Desarrollo Multiplataforma con React Native

Día 3: Navegación + Estructura de app

Carlos Rodríguez Contreras
carlos.mailhub@gmail.com
karrocon

23 de abril de 2026

De una pantalla a toda una app

Agenda del día

- Repaso Día 2
- ¿Qué es React Navigation?
- Stack Navigator y pantallas
- Pasar parámetros
- Opciones de cabecera
- Estructura de proyecto
- Ejercicios
- Cierre

Objetivo del día

Conectar **varias pantallas** con React Navigation.

Teoría breve → práctica inmediata.

Horario orientativo

16:00 Repaso
16:20 ¿Qué es React Nav? → mini-ej
17:00 Stack Nav + pantallas → mini-ej
18:00 **DESCANSO 30 min**
18:30 Pasar parámetros → mini-ej
19:00 Opciones cabecera → mini-ej
19:25 Estructura `src/` → mini-ej
19:55 **PARADA: ejercicios**
20:25 Cierre 20:30 Fin

Repaso del Día 2

Lo que ya sabemos

Repaso del Día 2

16:00 — 16:20 Repaso + preguntas

Componente completo del Día 2:

```
type ItemProps = {
  texto: string; hecho: boolean;
};
function Item({ texto, hecho }: ItemProps) {
  return (
    <Text style={{
      textDecorationLine:
        hecho ? 'line-through' : 'none',
    }}>
      {texto}
    </Text>
  );
}
// En App.tsx:
const [lista, setLista] =
  useState<string[]>(['Estudiar']);
{lista.map((t, i) =>
  <Item key={i} texto={t} hecho={false} />
)}
```

Cubierto el Día 2:

- ✔ Props tipadas con type
- ✔ Estado con useState
- ✔ Inputs controlados
- ✔ Renderizado condicional
- ✔ Listas con map() y key
- ✔ Estructura de proyecto

➜ Hoy empezamos

React Navigation
Varias pantallas
Pasar datos entre ellas
Estructura de proyecto real

¿Qué es React Navigation?

El estándar para navegar en React Native

¿Por qué necesitamos navegación?

En la web:

- El navegador gestiona el historial
- URL = pantalla
- Botón atrás nativo del navegador

En móvil:

- No hay URL ni navegador
- Necesitamos gestionar el historial a mano
- El botón «atrás» es nuestro

React Navigation es la biblioteca estándar que hace todo esto por nosotros.

Tipos de Navigator

- **Stack** → pantallas apiladas
  A → B → C, volver con ←
- **Tab** → pestañas inferiores
  Como Instagram/Twitter
- **Drawer** → menú lateral
  Como Gmail

📖 `reactnavigation.org`
Hoy solo usamos **Stack** ← el más frecuente

Instalación de React Navigation

16:20 — 17:00 React Navigation — instalación y primeros pasos

1. Dependencias del núcleo:

```
npm install @react-navigation/native
npm install @react-navigation/native-stack
```

2. Dependencias nativas (Expo):

```
npx expo install react-native-screens
npx expo install react-native-safe-area-context
```

⚠ `npx expo install` (no `npm install`) para estas dos: gestiona versiones compatibles con el SDK.

¿Qué instalo para qué?

- `@react-navigation/native`
  Núcleo: NavigationContainer
- `@react-navigation/native-stack`
  El Stack Navigator
- `react-native-screens`
  Rendimiento nativo
- `react-native-safe-area-context`
  Área segura (notch/barra)

Los ejercicios del repositorio ya tienen estas dependencias en `package.json`.
Solo necesitas hacer `npm install`.

NavigationContainer — el envoltorio raíz

App.tsx completo mínimo:

```
import { NavigationContainer }
  from '@react-navigation/native';
import { createNativeStackNavigator }
  from '@react-navigation/native-stack';
import HomeScreen    from './src/screens/HomeScreen';
import DetalleScreen from './src/screens/DetalleScreen';

const Stack =
  createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Home"
          component={HomeScreen}
        />
        <Stack.Screen
          name="Detalle"
          component={DetalleScreen}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Reglas:

- **Un solo** `NavigationContainer` en toda la app
- Debe envolver todo el árbol de navegación
- La primera `Screen` es la pantalla inicial

Analogía

`NavigationContainer` = el propio navegador web.

`Stack.Navigator` = el gestor de historial.

`Stack.Screen` = una pestaña registrada.

Mini-ejercicio — Instalar y conectar 2 pantallas

</> Mini-ejercicio (15 min)

Partiendo del proyecto del Día 2 (o uno nuevo):

- Instala React Navigation con los 4 paquetes
- Crea un `src/screens/HomeScreen.tsx` con un texto y un botón
- Crea un `src/screens/DetalleScreen.tsx` con un texto simple
- En `App.tsx`, añade el `NavigationContainer` y el `Stack.Navigator` con las dos pantallas
- Verifica que la cabecera aparece y que el título dice «Home» y «Detalle»

Bonus: cambia el nombre del título con `options={{ title: '...' }}`.

Stack Navigator y pantallas

Navegar, volver y tipar las rutas

TypeScript — RootStackParamList

17:00 — 18:00 Stack Navigator + pantallas

src/types/navigation.ts

```
import type { NativeStackScreenProps }
  from '@react-navigation/native-stack';

// Mapa de rutas y sus par metros:
export type RootStackParamList = {
  Home:     undefined;            // sin params
  Detalle: {                      // params tipados
    nombre: string;
    rol:    string;
  };
  Ajustes: undefined;
};

// Props de cada pantalla:
export type HomeProps =
  NativeStackScreenProps<
    RootStackParamList, 'Home'>;

export type DetalleProps =
  NativeStackScreenProps<
    RootStackParamList, 'Detalle'>;
```

¿Qué ganamos con TypeScript aquí?

- Autocompletado de nombres de pantallas
- Error si navegas a una ruta que no existe
- Error si falta un parámetro obligatorio
- `route.params` es seguro de tipar

Analogía Java

`RootStackParamList` ≈ `enum Routes` con tipos asociados.

`undefined` = pantalla sin params.

navigation.navigate() y navigation.goBack()

HomeScreen.tsx:

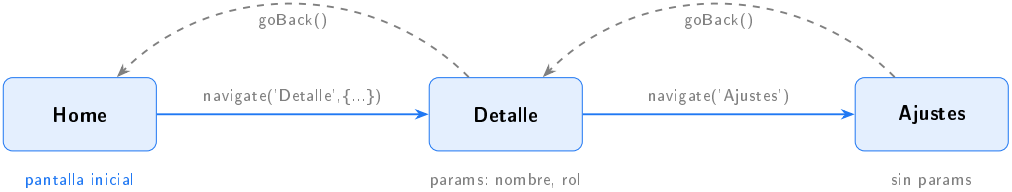
```
import type { HomeProps }
  from '../types/navigation';

export default function HomeScreen(
    { navigation }: HomeProps) {
  return (
    <View style={s.container}>
      <Text>Pantalla de inicio</Text>

      <TouchableOpacity
        style={s.btn}
        onPress={() =>
          navigation.navigate(
            'Detalle',
            { nombre: 'Ana', rol: 'Dev' }
          )
        }
      >
        <Text>Ver detalle    </Text>
      </TouchableOpacity>

      <TouchableOpacity
        onPress={() =>
          navigation.navigate('Ajustes')}
      >
        <Text>    Ajustes</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Métodos del objeto `navigation`:

- `navigate('NombrePantalla')`
  Va a la pantalla
- `navigate('P', {params})`
  Va y pasa datos
- `goBack()`
  Vuelve atrás (como ←)
- `push('P')`
  Apila aunque ya esté abierta
- `replace('P')`
  Reemplaza sin añadir al historial
- `popToTop()`
  Vuelve a la primera pantalla

La cabecera ya incluye un botón ← automático. `goBack()` hace lo mismo por código.

Cómo funciona el Stack — visualmente



El Stack funciona como una **pila de cartas**: cada `navigate` pone una encima, `goBack` la quita.

La cabecera es automática. El botón ← aparece en cuanto hay una pantalla debajo.

Mini-ejercicio — Navegar entre 3 pantallas

</> Mini-ejercicio (15 min)

Añade una tercera pantalla y practica la navegación:

- Crea `AjustesScreen.tsx` con un texto simple
- Registra las 3 pantallas en `App.tsx`
- En `HomeScreen` añade un botón «Ir a Ajustes»
- En `DetalleScreen` añade un botón «Volver» con `goBack()`
- Comprueba que la cabecera muestra la flecha atrás automáticamente

Bonus: añade `popToTop()` en Ajustes para volver directamente al inicio aunque hayas pasado por Detalle.

Descanso — 30 minutos

18:00 — 18:30 Descanso

30 minutos

Al volver: pasar parámetros y opciones de cabecera

Pasar parámetros

Enviar datos de una pantalla a otra

route.params — leer los datos en destino

18:30 — 19:00 Pasar parámetros

DetalleScreen.tsx:

```
import type { DetalleProps }
  from '../types/navigation';
import { View, Text, StyleSheet }
  from 'react-native';

export default function DetalleScreen(
    { route, navigation }: DetalleProps) {
  // TypeScript sabe el tipo de params:
  const { nombre, rol } = route.params;

  return (
    <View style={s.container}>
      <Text style={s.avatar}>    </Text>
      <Text style={s.nombre}>{nombre}</Text>
      <Text style={s.rol}>{rol}</Text>
      <Text style={s.info}>
        Perfil de {nombre}, rol: {rol}
      </Text>
    </View>
  );
}
const s = StyleSheet.create({
  container: { flex: 1,
    alignItems: 'center',
    justifyContent: 'center' },
  nombre: { fontSize: 28,
    fontWeight: 'bold' },
  rol: { fontSize: 18, color: '#666' },
  avatar: { fontSize: 64 },
  info: { marginTop: 20 },
```

Flujo completo:

```
// En HomeScreen:
navigation.navigate(
  'Detalle',
  { nombre: 'Ana',
    rol: 'Dev' }
);
```

```
// En DetalleScreen:
const { nombre, rol }
  = route.params;
```

⚠ Los params son **solo lectura**. No los modifiques directamente — usa estado local si necesitas editar.

Pasar ID y cargar datos en destino

Patrón habitual: pasar solo el ID

```
// En el tipo:
type RootStackParamList = {
  Lista:   undefined;
  Detalle: { id: number };
};

// Desde Lista:
navigation.navigate(
  'Detalle', { id: pokemon.id }
);

// En Detalle:
const { id } = route.params;

useEffect(() => {
  fetch(`https://pokeapi.co/api/v2/
    pokemon/${id}`)
    .then(r => r.json())
    .then(setDatos);
}, [id]);
```

Dos estrategias:

Pasar todos los datos

- ✔ Más sencillo
- ✔ No necesita más fetch
- ✖ Sobrecarga el historial

Pasar solo el ID

- ✔ Pantalla independiente
- ✔ Siempre datos frescos
- ✔ Mejor para producción

En los días 5-6 usaremos el patrón ID + fetch.

Mini-ejercicio — Pasar datos entre pantallas

</> Mini-ejercicio (15 min)

Actualiza tu app del mini-ejercicio anterior:

- En `HomeScreen` crea una lista de 3 personas con `nombre`, `rol` e `id`
- Al pulsar cada persona, navega a `DetalleScreen` pasando `nombre` y `rol`
- En `DetalleScreen` muestra los datos con un diseño básico (avatar emoji, nombre grande, rol en gris)
- Verifica que TypeScript se queja si intentas navegar sin pasar los params

Bonus: añade un parámetro `ciudad?: string` (opcional) y muéstralo solo si existe.

Pista: `ciudad?: string` en el tipo; en destino: `{ciudad && <Text>{ciudad}</Text>}`

Opciones de cabecera

Personalizar el header de cada pantalla

screenOptions vs options

19:00 — 19:25 Opciones de cabecera

screenOptions: afecta a TODAS las pantallas

```
<Stack.Navigator
  screenOptions={{
    headerStyle: {
      backgroundColor: '#1a73e8',
    },
    headerTintColor: 'white',
    headerTitleStyle: {
      fontWeight: 'bold',
    },
  }}
>
```

options: afecta solo a una pantalla

```
<Stack.Screen
  name="Home"
  component={HomeScreen}
  options={{ title: 'Directorio' }}
/>

// T tulo din mico (desde params):
<Stack.Screen
  name="Detalle"
  component={DetalleScreen}
  options={({ route }) => ({
    title: route.params.nombre,
  })}
/>
```

Opciones más usadas:

- `title` — texto de la cabecera
- `headerStyle` — estilo del fondo
- `headerTintColor` — color de icono y texto
- `headerTitleStyle` — estilo del título
- `headerShown: false` — oculta la cabecera
- `headerRight` — elemento derecho
- `headerLeft` — elemento izquierdo

💡 Con `options={({route}) => ...}` tienes acceso a los params para títulos dinámicos.

Botón personalizado en la cabecera

```
<Stack.Screen
  name="Home"
  component={HomeScreen}
  options={({ navigation }) => ({
    title: 'Directorio',
    headerRight: () => (
      <TouchableOpacity
        onPress={() =>
          navigation.navigate('Ajustes')
        }
        style={{ marginRight: 8 }}
      >
        <Text style={{ color: 'white',
          fontSize: 20 }}>   </Text>
      </TouchableOpacity>
    ),
  })}
/>
```

Cuándo usarlo

headerRight: botón de acción (ajustes, info, guardar)

headerLeft: reemplazar botón atrás por uno propio

⚠ En iOS el botón ← incluye el título de la pantalla anterior. Puedes cambiarlo con `headerBackTitle: ''`.

📖 `reactnavigation.org/docs`

Mini-ejercicio — Cabecera de colores

</> Mini-ejercicio (10 min)

Personaliza la cabecera de tu app:

- Cambia el fondo del header a un color a tu gusto con `screenOptions`
- En `DetalleScreen`, muestra el nombre de la persona como título dinámico
- Añade un botón «» en la cabecera de `HomeScreen` que navegue a Ajustes

Bonus: oculta la cabecera en `AjustesScreen` con `headerShown: false` y pon un botón de cerrar propio.

Estructura de proyecto con navegación

Cómo organizar una app real

Estructura recomendada para una app con navegación

19:25 — 19:55 Estructura de proyecto

```
miApp/
  App.tsx ← solo navegación
  src/
    screens/
      HomeScreen.tsx
      DetalleScreen.tsx
      AjustesScreen.tsx
    components/
      Tarjeta.tsx
      Boton.tsx
    types/
      navigation.ts
    constants/
      colors.ts
  assets/
```

Principios

App.tsx solo monta el navegador.

screens/ una pantalla = un archivo.

components/ piezas reutilizables.

types/navigation.ts único punto de verdad para las rutas.

constants/colors.ts evita colores hardcodeados.

Si `App.tsx` tiene más de 30 líneas, algo de lógica debería estar en una screen o componente.

App.tsx ideal — solo navegación

App.tsx limpio:

```
import { NavigationContainer }
  from '@react-navigation/native';
import { createNativeStackNavigator }
  from '@react-navigation/native-stack';

import type { RootStackParamList }
  from './src/types/navigation';
import HomeScreen     from './src/screens/HomeScreen';
import DetalleScreen from './src/screens/DetalleScreen';
import AjustesScreen from './src/screens/AjustesScreen';

const Stack =
  createNativeStackNavigator<
    RootStackParamList>();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator
        screenOptions={{
          headerStyle:      { backgroundColor: '#1a73e8' },
          headerTintColor: 'white',
        }}
      >
        <Stack.Screen name="Home"
          component={HomeScreen}
          options={{ title: 'Directorio' }}
        />
        <Stack.Screen name="Detalle"
          component={DetalleScreen}
          options={({ route }) =>
            ({ title: route.params.nombre })}
        />
        <Stack.Screen name="Ajustes"
          component={AjustesScreen}
          options={{ title: 'Ajustes' }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

✔ Qué tiene este App.tsx

1. Importa tipos de `src/types/`
2. Stack tipado con `RootStackParamList`
3. `screenOptions` globales
4. Título dinámico desde `params`
5. Cero lógica de negocio

⚠ No pongas `useState` en `App.tsx` si ya tienes navegación. Cada pantalla gestiona su propio estado.

Mini-ejercicio — Refactoriza tu app

</> Mini-ejercicio (10 min)

Aplica la estructura correcta a tu app:

- Mueve las pantallas a `src/screens/`
- Crea `src/types/navigation.ts` con `RootStackParamList`
- Deja `App.tsx` con solo el NavigationContainer y los Stack.Screen
- Extrae los colores del header a `src/constants/colors.ts`

Bonus: crea un componente `src/components/FilaPersona.tsx` que sea la tarjeta de cada persona en HomeScreen.

Ejercicios en clase

Dos retos para cerrar el día

Ejercicio 1 — Directorio con 3 pantallas

Enunciado

App con **3 pantallas**:

- ✔ **HomeScreen** — lista de personas (FlatList)
- ✔ **DetalleScreen** — perfil completo (nombre, rol, avatar)
- ✔ **AjustesScreen** — texto simple

Requisitos:

- Cabecera azul en todas
- Título dinámico en Detalle
- Botón Ajustes en cabecera de Home

Bonus: añade `src/components/FilaPersona.tsx`

dia-03/ejercicios/ej01-navegacion/

Estructura de partida:

```
// src/types/navigation.ts
export type RootStackParamList = {
  Home:     undefined;
  Detalle: { nombre: string; rol: string };
  Ajustes: undefined;
};

// src/screens/HomeScreen.tsx
const PERSONAS = [
  { id: '1', nombre: 'Ana Garc a',
    rol: 'Dise adora' },
  { id: '2', nombre: 'Carlos L pez',
    rol: 'Desarrollador' },
  { id: '3', nombre: 'Marta Ruiz',
    rol: 'Product Manager' },
];
```

Ejercicio 2 — Proto-Pokédex

Enunciado

App de **2 pantallas** con datos hardcodeados:

✔ **ListaScreen** — grid 2 columnas (FlatList) con sprite, número y nombre

✔ **DetalleScreen** — sprite grande, nombre, número y tipo

Requisitos:

- Cabecera roja
- `numColumns={2}` en FlatList
- Navegación con paso de params

Bonus: badge de color según tipo (planta, fuego, agua)

dia-03/ejercicios/ej02-proto-pokedex/

Datos de partida:

```
type Pokemon = {
  id: number; nombre: string;
  tipo: string;
};

const POKEMON: Pokemon[] = [
  { id: 1,  nombre: 'Bulbasaur',
    tipo: 'planta' },
  { id: 4,  nombre: 'Charmander',
    tipo: 'fuego'  },
  { id: 7,  nombre: 'Squirtle',
    tipo: 'agua'   },
  { id: 25, nombre: 'Pikachu',
    tipo: 'el ctrico' },
  { id: 39, nombre: 'Jigglypuff',
    tipo: 'normal' },
  { id: 52, nombre: 'Meowth',
    tipo: 'normal' },
];

const sprite = (id: number) =>
  `https://raw.githubusercontent.com/
PokeAPI/sprites/master/sprites/
pokemon/${id}.png`;
```

Parada: Ejercicios en clase

19:55 — 20:25 Trabajo autónomo guiado

Los alumnos trabajan en los 2 ejercicios.

El profesor:

- Rota por los puestos
- Ayuda con errores de tipos (TypeScript)
- Ojo: error frecuente = olvidar tipos en `RootStackParamList`
- Muestra solución en proyector si hay bloqueo general

Ejercicios en: `dia-03/ejercicios/`

Criterio de éxito

- **Ej1:** 3 pantallas navegables + params
- **Ej2:** grid de Pokémon + detalle al pulsar

Bonus: FilaPersona / badge de tipo

Error más común:
`undefined` en `route.params`
→ revisar si la Screen está registrada en el Stack

Resumen del día

Hoy hemos aprendido:

- ✔ Instalar y configurar **React Navigation**
- ✔ **NavigationContainer** + `Stack.Navigator`
- ✔ Registrar pantallas con `Stack.Screen`
- ✔ Navegar con `navigation.navigate()` y `goBack()`
- ✔ Tipar rutas con **RootStackParamList**
- ✔ Pasar y leer **parámetros** con `route.params`
- ✔ Personalizar la **cabecera** (color, título dinámico)
- ✔ Estructura de proyecto con `src/`

📅 Día 4 — Próxima sesión

- **Consumo de APIs** con `fetch`
- `async/await` y promesas
- `useEffect` para cargar datos
- Manejo de estado: loading / error / datos
- Primera llamada a **PokéAPI**

➜ La Pokédex empieza a tener datos reales

Recursos y referencias

Documentación oficial:

- `reactnavigation.org` — guía principal
- `.../native-stack-navigator` — opciones
- `.../params` — pasar params
- `reactnative.dev/navigation` — visión general

Playground online:

- `snack.expo.dev` — prueba en el navegador

Para profundizar:

- `.../tab-based-navigation` — Tab Navigator
- `.../drawer-based-navigation` — Drawer
- `.../typescript` — guía TypeScript completa

Usa **Snack** para probar la navegación sin instalar nada. Comparte con una URL.

`karrocon/taller-react-native-fp`
Ejercicios y soluciones del curso

¿Preguntas?

Antes de salir:

✔ Sabes registrar pantallas en un Stack
✔ Puedes pasar datos con `navigation.navigate()`
✔ Lees los params con `route.params`
✔ Tienes tipados los params con `RootStackParamList`

Si no es así → **ahora es el momento de preguntar**